

laboratorio-resolvido-semana-4-v3

August 25, 2026

1 Aula 4: Pré-processamento e Transformação Estrutural

1.1 Guia de Laboratório

Este notebook aborda as 4 fases: 1. **Distribuição Normal / Gaussiana** (álgebra e vetorização no NumPy). 2. **Limpeza e Pré-processamento** (Tratamento de NaNs e duplicatas). 3. **Pandas GroupBy** (Estratégia Dividir-Aplicar-Combinar aplicada ao Titanic).

1.1.1 Fase 1: Fixação Local (NumPy de Baixo Nível)

Vamos gerar amostras que obedecem a uma distribuição Normal de forma manual e verificar sua média e variância de próprio punho.

```
[1]: import numpy as np
import matplotlib.pyplot as plt

# Fixa semente estatística para manter reprodutibilidade dos dados
np.random.seed(2022)

# 1. Geração de 500 amostras simuladas de uma distribuição Gaussiana
# Definimos a média como 10.0 e o desvio padrão como 2.0
mu_real = 10.0
sigma_real = 2.0
x_amostras = np.random.normal(loc=mu_real, scale=sigma_real, size=500)

# 2. Funções manuais para calcular a média e desvio padrão amostral
def calcular_media_manual(vetor):
    # Soma todos os termos e divide pelo tamanho do array
    return np.sum(vetor) / len(vetor)

def calcular_std_manual(vetor, media):
    # Calcula desvio padrão amostral utilizando graus de liberdade n-1
    soma_desvios_quad = np.sum((vetor - media) ** 2)
    return np.sqrt(soma_desvios_quad / (len(vetor) - 1))

media_calc = calcular_media_manual(x_amostras)
std_calc = calcular_std_manual(x_amostras, media_calc)

print(f"Média Calculada: {media_calc:.4f} (Real: {mu_real})")
```

```
print(f"Desvio Padrão Calculado: {std_calc:.4f} (Real: {sigma_real})")
```

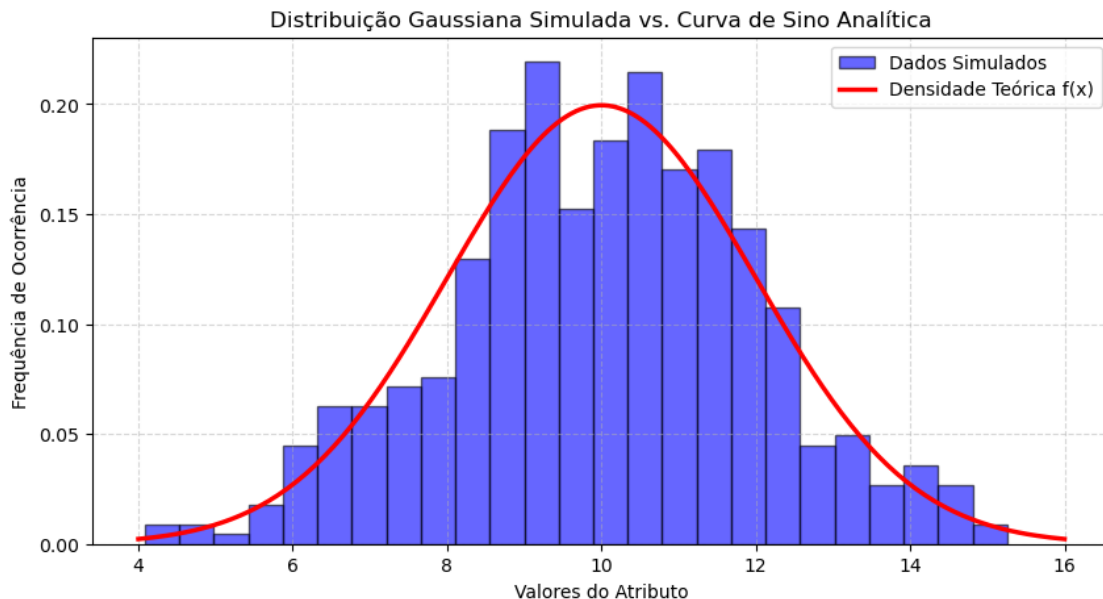
Média Calculada: 10.0047 (Real: 10.0)

Desvio Padrão Calculado: 2.0014 (Real: 2.0)

```
[2]: # 3. Plotagem do histograma versus densidade analítica teórica
def densidade_gaussiana_teorica(x, mu, sigma):
    # Formulação matemática exata  $f(x)$  da distribuição normal
    termo_escala = 1.0 / (sigma * np.sqrt(2 * np.pi))
    expoente = -0.5 * ((x - mu) / sigma) ** 2
    return termo_escala * np.exp(expoente)

x_intervalo = np.linspace(4, 16, 200)
y_densidade = densidade_gaussiana_teorica(x_intervalo, mu_real, sigma_real)

plt.figure(figsize=(10, 5))
plt.hist(x_amostras, bins=25, density=True, alpha=0.6, color='blue',
        edgecolor='black', label='Dados Simulados')
plt.plot(x_intervalo, y_densidade, color='red', linewidth=2.5, label='Densidade
        Teórica  $f(x)$ ')
plt.title('Distribuição Gaussiana Simulada vs. Curva de Sino Analítica')
plt.xlabel('Valores do Atributo')
plt.ylabel('Frequência de Ocorrência')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



1.1.2 Fase 2: Integração Guiada (Pandas, Titanic e Bloco de Erro)

Vamos simular e interagir com o dataset do Titanic carregado via Pandas, estudando duplicados e tratando erros de barramento de tipos.

```
[3]: import pandas as pd

# Criamos um DataFrame simulando dados reais ruidosos do Titanic
dados_titanic = {
    'PassengerId': [1, 2, 3, 4, 5, 2, 6, 7],
    'Survived': [0, 1, 1, 1, 0, 1, 0, 1],
    'Pclass': [3, 1, 3, 1, 3, 1, 2, 3],
    'Sex': ['male', 'female', 'female', 'female', 'male', 'female', 'male', 'female'],
    'Age': ['22.0', '38.0', '26.0', '35.0', 'NULL', '38.0', '29.0', 'NULL'],
    'Fare': [7.25, 71.28, 7.92, 53.10, 8.05, 71.28, 13.00, 8.05]
}
df_raw = pd.DataFrame(dados_titanic)
print("DataFrame Original com Redundâncias e Strings Nulas:")
print(df_raw)
```

DataFrame Original com Redundâncias e Strings Nulas:

	PassengerId	Survived	Pclass	Sex	Age	Fare
0	1	0	3	male	22.0	7.25
1	2	1	1	female	38.0	71.28
2	3	1	3	female	26.0	7.92
3	4	1	1	female	35.0	53.10
4	5	0	3	male	NULL	8.05
5	2	1	1	female	38.0	71.28
6	6	0	2	male	29.0	13.00
7	7	1	3	female	NULL	8.05

BLOCO DE ERRO CONTROLADO (Traceback) Veja o erro acontecer ao tentar calcular a média de uma coluna de texto (tipo object) que contém o valor 'NULL' em vez de números.

```
[4]: try:
    # Isto falhará ou não calculará a média desejada, pois a coluna 'Age' é do
    # tipo texto
    print("Tentando calcular a média direta da coluna ruidosa:")
    media_ruidosa = df_raw['Age'].mean()
except TypeError as e:
    print("\n--- TRACEBACK CONTROLADO ---")
    print(f"TypeError detectado com sucesso: {e}")
    print("Explicativo: A CPU não consegue computar desvios matemáticos sobre
    strings. Devemos coagir tipos primeiro.")
```

Tentando calcular a média direta da coluna ruidosa:

--- TRACEBACK CONTROLADO ---

TypeError detectado com sucesso: Could not convert
22.038.026.035.0NULL38.029.0NULL to numeric
Explicativo: A CPU não consegue computar desvios matemáticos sobre strings.
Devemos coagir tipos primeiro.

```
[5]: # Solução estruturada e limpeza do DataFrame
df_clean = df_raw.copy()

# 1. Elimina duplicados utilizando o indexador PassengerId como referência
df_clean = df_clean.drop_duplicates(subset=['PassengerId'], keep='first')

# 2. Converte tipos coercitivamente, mapeando ruídos como NaNs legítimos
df_clean['Age'] = pd.to_numeric(df_clean['Age'], errors='coerce')

print("DataFrame Higienizado e Tipado:")
print(df_clean)
```

DataFrame Higienizado e Tipado:

	PassengerId	Survived	Pclass	Sex	Age	Fare
0	1	0	3	male	22.0	7.25
1	2	1	1	female	38.0	71.28
2	3	1	3	female	26.0	7.92
3	4	1	1	female	35.0	53.10
4	5	0	3	male	NaN	8.05
6	6	0	2	male	29.0	13.00
7	7	1	3	female	NaN	8.05

1.1.3 Fase 3: Verificação de Pressupostos (Diagnóstico Estatístico)

Vamos testar as implicações estatísticas de duas estratégias de imputação de nulos na distribuição de probabilidade da coluna 'Age'.

```
[6]: # Geração de dados de idade simulados com nulos para diagnóstico profundo
np.random.seed(42)
idades_originais = np.random.normal(loc=30, scale=8, size=100)
df_diag = pd.DataFrame({'Age': idades_originais})

# Adiciona 20% de nulos de forma artificial
df_diag.loc[df_diag.sample(frac=0.2).index, 'Age'] = np.nan

# Estratégia A: Imputação univariada pela média global
df_media_global = df_diag.copy()
df_media_global['Age'] = df_media_global['Age'].fillna(df_media_global['Age'].
↳ mean())

# Imprime a contração matemática da variância
var_original = df_diag['Age'].var() # calcula ignorando nulos
var_imputada_media = df_media_global['Age'].var()
```

```

print(f"Variância Amostral Original (Sem Nulos): {var_original:.4f}")
print(f"Variância Amostral Pós-Imputação por Média: {var_imputada_media:.4f}")
print(f"Contração de Variância calculada de: {((var_original -
↪var_imputada_media)/var_original)*100:.2f}%")

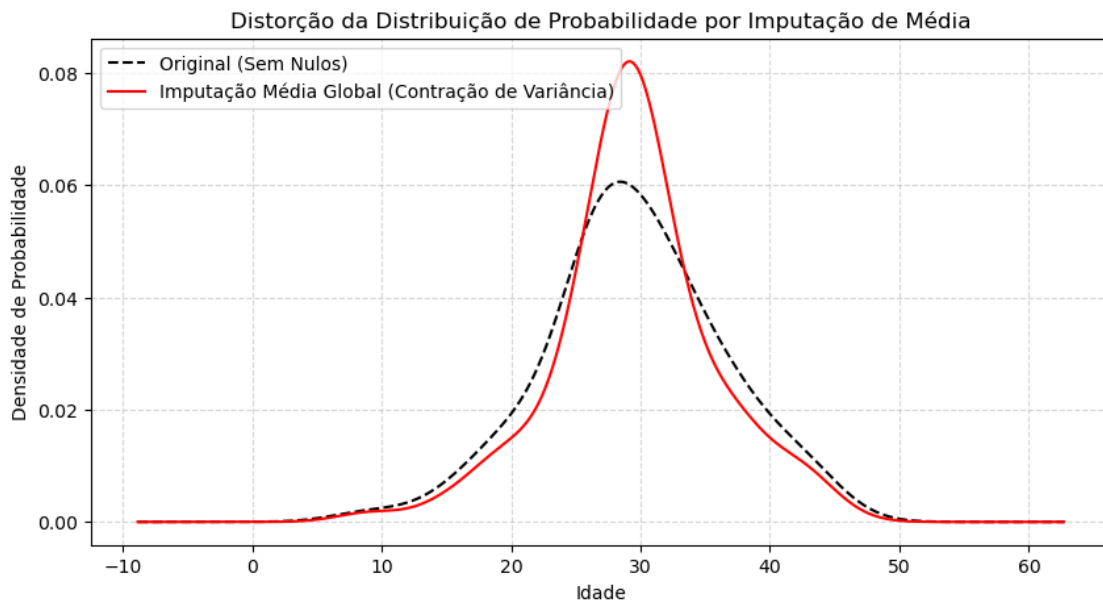
```

Variância Amostral Original (Sem Nulos): 46.7934
 Variância Amostral Pós-Imputação por Média: 37.3402
 Contração de Variância calculada de: 20.20%

```

[7]: # Desenha gráfico de densidade de probabilidade comparativo
plt.figure(figsize=(10, 5))
df_diag['Age'].plot(kind='kde', color='black', style='--', label='Original (Sem
↪Nulos)', ax=plt.gca())
df_media_global['Age'].plot(kind='kde', color='red', label='Imputação Média
↪Global (Contração de Variância)', ax=plt.gca())
plt.title('Distorção da Distribuição de Probabilidade por Imputação de Média')
plt.xlabel('Idade')
plt.ylabel('Densidade de Probabilidade')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

```



1.1.4 Fase 4: Desafio de Sistema (GroupBy e Equivalência Matemática)

Vamos realizar uma imputação baseada em subgrupos utilizando groupby condicional e provar que nossa regra corresponde a agregação local.

```
[8]: # Carrega dataframe com as idades nulas e classes de passagem
df_sistema = df_clean.copy()

# 1. Agrupamento pelo critério de classe para encontrar a mediana de cada grupo
grupo_mediana = df_sistema.groupby('Pclass')['Age'].transform('median')

# 2. Imputação refinada com base no contexto do passageiro
df_sistema['Age'] = df_sistema['Age'].fillna(grupo_mediana)

# 3. Provando a equivalência matemática por checagem condicional
medianas_reais = df_clean.groupby('Pclass')['Age'].median()
print("Medianas calculadas locais dos grupos (Pclass):")
print(medianas_reais)

print("\nDataFrame de Saída Tratado com Mediana Condicional:")
print(df_sistema)
```

Medianas calculadas locais dos grupos (Pclass):

Pclass

```
1    36.5
2    29.0
3    24.0
```

Name: Age, dtype: float64

DataFrame de Saída Tratado com Mediana Condicional:

	PassengerId	Survived	Pclass	Sex	Age	Fare
0	1	0	3	male	22.0	7.25
1	2	1	1	female	38.0	71.28
2	3	1	3	female	26.0	7.92
3	4	1	1	female	35.0	53.10
4	5	0	3	male	24.0	8.05
6	6	0	2	male	29.0	13.00
7	7	1	3	female	24.0	8.05

[]: